

Volatile Demystified

In C programming, the use of the key word 'volatile' is intended to prevent the compiler from applying any optimisations on objects that can be changed in ways that the compiler cannot determine. In this article, the author demystifies the use of the volatile key word.

```
int main()
{
    volatile int i;

    //Some code goes
    here

    //Loop for delay
    generation
    for( i = 0; i <
    100; i++)
    {
        ;
    }

    //Again the
    remaining code
    goes here

    return 0;
}
```

```
int main()
{
    int i = 0;
    int flag = 0;

    //Some code

    while(TRUE)
    {
        //Infinite loop
        ;
    }
    //rest of the code
    return 0;
}
```

Volatile is a qualifier in C, which is applied to a variable when it is declared. It is used extensively while writing programs for embedded systems, especially when dealing with hardware. The aim of this article is to give you an idea of how to use a Volatile qualifier. So, what are its instructions to the compiler? It tells the compiler that the value of the variable may change at any time during the execution of the code without the knowledge of the compiler. If proper precautions are not taken, the desired output may not be achieved. A variable should be declared volatile whenever its value may change unexpectedly.

Volatile variables are, therefore, variables that can be changed at any time by other external programs or by the same program.

The syntax for declaring the variable as 'volatile' is:

```
volatile dataType variable;
```

Let us understand the 'volatile' key word thoroughly through the following illustrations.



Note: Remember that all codes are compiled in the gcc compiler version: gcc version 4.8.2 (Ubuntu 4.8.2-19ubuntu1)

Illustration 1: Polling mechanism

Let us consider the following small and simple code snippet to study the behaviour of the 'volatile' key word in C:

```
1 int main()
2 {
3     int i = 0;
4     int flag = 0;
5
6     //Some code
7
8     while(flag != 1)
9     {
10         //Keep polling until flag becomes 1
11         ;
12     }
13
14     //rest of the code
15     return 0;
16 }
```

Here, the intention of the programmer is to keep polling inside the *while* loop until the flag value is SET to the value 1, which might be done by a hardware/peripheral device. However, during the compilation phase, the compiler will find